

基于LLM的灾害人群疏散多智能体建模 —— LLM→IRL→RL 三级联级架构

报告提纲

- 问题背景与核心挑战
- 现有方法对比与技术选型
- 系统架构总览
- 重点模块详解
- 实验设计与当前进展
- 诚实讨论：局限性 & 后续工作

一、问题背景与核心挑战

应用场景

武汉保利广场1F发生火灾，200m×120m商场内600人需在360秒内从6个出口（其中2个因火势封锁）疏散。核心问题：**如何调度人群使疏散效率最大化。**

传统方法的瓶颈

传统疏散仿真（如Pathfinder、FDS+Evac）使用Social Force Model (SFM)等物理模型，Agent按"最近出口+烟雾惩罚"的规则行动。问题在于：

- 所有Agent行为同质化——年轻人、老人、店员没有区别
- 没有"预测"能力——只看到当前烟雾，不知道火在蔓延
- 没有社交因素——家人不会互相等待，人群不会跟随引导员

为什么需要LLM

LLM Agent能做三件传统模型做不到的事：

- 常识推理**："我的位置烟雾很浓，虽然出口B更近，但它在火源方向上，我选远的出口A"
- 角色差异化**：店员知道员工通道，老人走不动需要帮助，消防员主动救人
- 理解和响应指令**：引导员喊"跟我走！"，附近Agent可以选择跟随

核心创新：LLM → IRL → RL 三级联级

绝大多数LLM+RL组合论文的做法是"LLM和RL各自出决策然后加权平均"，这本质上只是两个现有技术的拼凑。

我们的方案不同：**LLM教会RL什么是好的决策，然后RL去执行调度。**IRL（逆强化学习）是中间的桥梁——从LLM的多样化行为中提取"隐含的价值观"，转化为RL可以直接优化的奖励函数。

LLM行为数据 → IRL提取奖励权重 → RL优化区域调度 → RL建议文本 → 注入LLM Prompt

↑

闭环：RL建议帮助LLM做更好的决策

二、技术选型对比

2.1 LLM推理引擎：vLLM vs DeepSpeed vs llama.cpp

维度	vLLM (选用)	DeepSpeed	llama.cpp
显存效率	PagedAttention, ~2GB (AWQ 4-bit)	~6GB (ZeRO-3)	~3GB (GGUF)
批量推理	Continuous batching, 自动合并	手动管理micro-batch	无原生batching
Prefix Caching	内置, 相同System Prompt只编码一次	需手动实现	不支持
600 Agent场景适用性	优秀 — 异步批量推理, GPU利用率>80%	一般 — 设计为训练框架	差 — 无批量, 600次独立推理
部署复杂度	中等, pip install	高, 需配置ZeRO策略	低, 单文件

选择理由：我们的场景是600个Agent共享同一个System Prompt，只是User Prompt不同（各自位置、烟雾、恐惧状态不同）。vLLM的Prefix Caching让这600条请求共享System Prompt的KV Cache，显存和延迟都大幅降低。

2.2 LLM模型：Qwen2.5-7B vs Llama-3 vs 闭源API

维度	Qwen2.5-7B-AWQ (选用)	GPT-4o API	Llama-3-8B
中文能力	原生优秀	需要prompt调优	一般
推理速度	~50 tokens/s (4090)	~100 tokens/s (但网络延迟)	~45 tokens/s
成本	0 (本地部署)	\$0.015/1K tokens × 600 Agent × 360s ≈ 昂贵	0
可复现性	完全可复现	API版本可能变化	可复现
可控性	可微调LoRA	只可prompt	可微调

选择理由：学术论文要求可复现、零成本跑大量实验、中文场景支持好。Qwen2.5-7B在中文指令跟随上优于同规模开源模型。

2.3 IRL算法：MaxEnt vs 学徒学习 (Apprenticeship) vs 贝叶斯IRL

维度	GC-MaxEnt (选用)	学徒学习	贝叶斯IRL
核心思想	最大熵原则：行为分布不确定性最大	直接匹配专家特征期望	对权重施加先验分布
对噪声的鲁棒性	好 — 不强制精确匹配	差 — 要求专家是最优的	中等
多模态行为处理	自然 — 最大熵允许行为多样性	差 — 只有一个"最优"策略	中等
计算复杂度	$O(N_{states} \times N_{actions} \times VI_{iters})$	较简单 (QP)	高 (采样)
跨人设扩展性	GC-MaxEnt：联合优化+Laplacian 正则化	需独立拟合每个人设	可层级建模

选择理由：

- 最大熵假设适合人类行为：** 真实疏散中，不是所有人都选最优路线，有人从众、有人犹豫、有人找家人。最大熵原则（"不确定性最大时，所有行为等概率"）比"单峰最优"更符合实际。
- GC正则化防止策略混淆：** 5种人设（老人/年轻人/店员/引导员/消防员）数据量不均衡（老人数据少），正则化项让相似人设的权重"借力"，防止小样本过拟合。

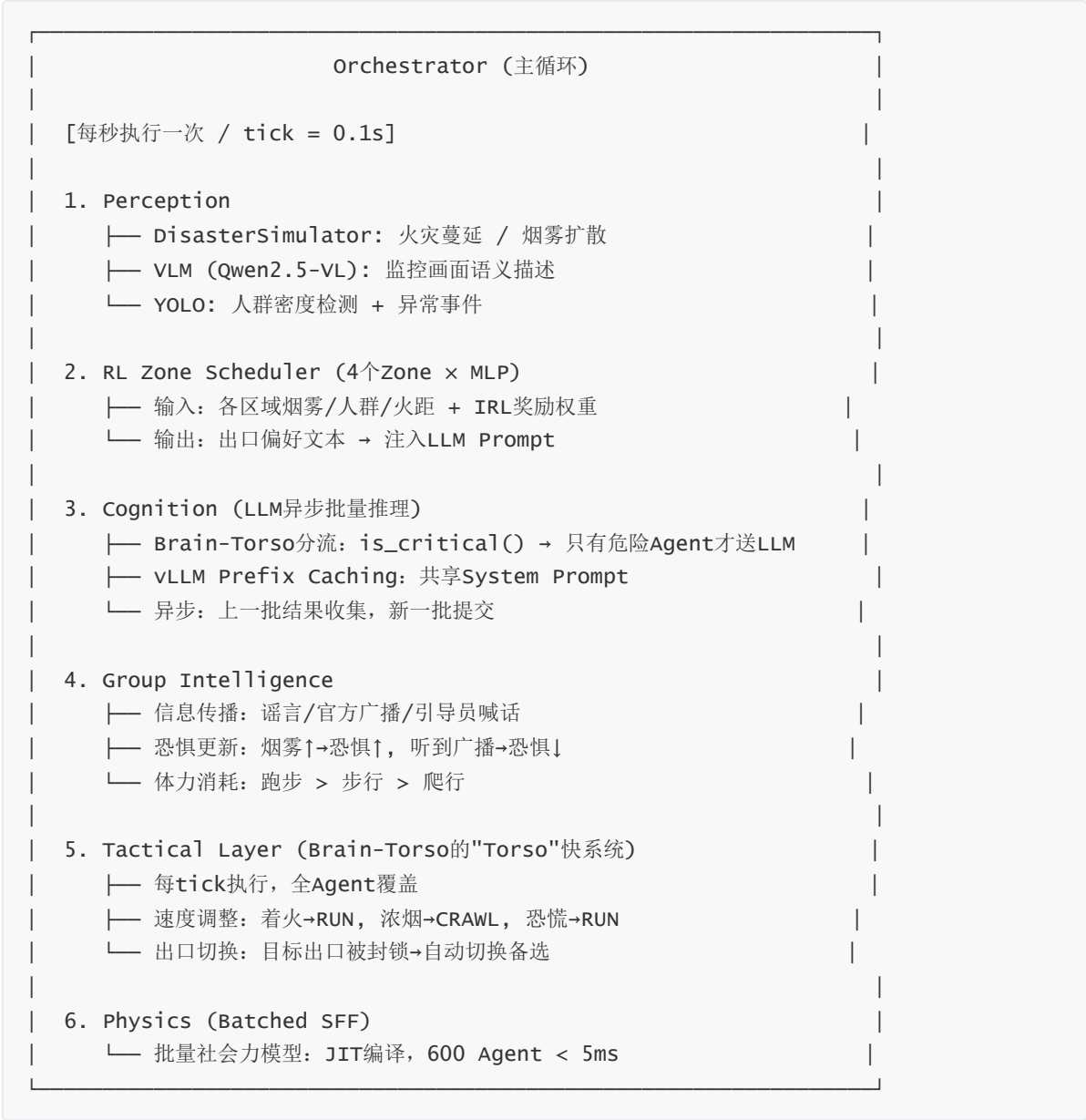
2.4 RL调度：P-MAPPO (多智能体PPO) vs 集中式DQN vs OR-Tools 数学规划

维度	P-MAPPO (选用)	集中式DQN	OR-Tools整数规划
多智能体协调	4个Zone Agent同时决策	1个全局Agent 动作空间巨大	全局最优但是静态
推理速度	<1ms (MLP前向)	<1ms (网络推理)	慢 (NP-hard, 秒级)
场景动态性	适应实时烟雾、人群变化	需要状态离散化	不适应 — 每次重算
奖励函数来源	IRL权重 (从LLM行为学来)	手动定义	手动定义
训练难度	中等 (PPO+GAE)	中等	无 (不需要训练)

选择理由：

- 区域级而非个体级调度：** 不控制每个Agent的每一步，而是给每个Zone出"建议文本"。这既高效（4个MLP vs 600个策略），又保留了LLM的最终决策权（建议≠命令）。
- IRL奖励函数是核心差异：** 集中式DQN和OR-Tools都用"人写的奖励函数"，我们的奖励函数是从LLM行为中"学来的"，这在学术上有明确的创新点。

三、系统架构总览



四、重点模块详解

4.1 Oracle轨迹生成器 —

experiments/generate_oracle_trajectories.py

在三级架构中的角色：为IRL提供带"标准答案"的训练数据。

为什么需要Oracle

这是一个学术界诚实性的关键决策。直接拿LLM跑出来的轨迹训练IRL会遇到一个死循环：

- LLM的决策 ≈ SFM启发式 ("最近出口+烟雾惩罚")

→ IRL从这种轨迹中只能学到"人喜欢近的出口"

→ RL拿到这个reward也只能说"去近的出口"

→ 整个三级架构退化成一个复杂的最近出口计算器

Oracle打破了循环：它用火势预测做决策，产生safety和efficiency之间存在真实冲突的数据。

Oracle决策算法

```
# 对每个出口计算预测的到达时烟雾值
travel_time = dist / speed
fire_reach_exit_time = fire_dist_to_exit / spread_rate

if fire_reach_exit_time < travel_time * 0.5:
    smoke_at_arrival = 0.95 # 火先到, 极其危险
elif fire_reach_exit_time < travel_time * 1.5:
    progress = travel_time / fire_reach_exit_time
    smoke_at_arrival = smoke_now + progress * 0.6 # 线性内插
else:
    smoke_growth_rate = 1.0 / fire_reach_exit_time
    smoke_at_arrival = smoke_now + travel_time * smoke_growth_rate # 缓慢增长

# Oracle打分
score = safety_at_arrival * 0.45 + efficiency * 0.25
       - crowd_penalty * 0.15 - fire_risk * 0.15
```

与启发式的本质区别

场景: 出口A距离20m, 但在火源方向; 出口B距离80m, 但远离火源

启发式: $\text{score}(B) = -80 * (1 + 0 * 3) = -80$, $\text{score}(A) = -20 * (1 + 0 * 3) = -20$
→ 选A (最近的出口 = 最好的出口)

Oracle: 预测到达A时烟雾0.9, 到达B时烟雾0.05
 $\text{score}(B) = 0.95 * 0.45 + 0.2 * 0.25 - 0 - 0.03 = 0.45$
 $\text{score}(A) = 0.10 * 0.45 + 0.67 * 0.25 - 0 - 0.20 = 0.09$
→ 选B (绕远路保命, safety > efficiency)

诚实定位

Oracle数据的作用**不是**提供"真实人类数据"——没有人知道真实人类在火灾中的脑中权重。Oracle数据的作用是:

1. **验证IRL算法数学正确性**: 已知权重 $w = \langle 0.45, 0.25, -0.15, -0.15 \rangle$ 生成行为, IRL应能近似恢复这些权重。这是IRL论文的标准验证方法。
2. **提供特征冲突数据**: 普通启发式轨迹中safety和efficiency总是同向(最近的出口=最安全的出口), IRL学不到区分。Oracle轨迹中两者冲突, IRL才能反推出"这个人在安全 vs. 效率之间更看重安全"。

4.2 IRL逆强化学习 — `execution/irl_recovery.py`

在三级架构中的位置: LLM行为数据 → IRL权重 → RL奖励函数

算法: GC-MaxEnt (组约束最大熵逆强化学习)

基础MaxEnt IRL (Ziebart et al., 2008):

- 假设观测到的行为分布满足最大熵原则——"除了观测到的偏好, 没有其他隐含偏好"
- 目标: 找到权重 w 使专家特征期望 $\approx$ 策略特征期望

- 梯度: $\nabla w = \mu_{\text{expert}} - \mu_{\pi}(w)$

我们的改进 — GC正则化:

- 问题: 5种人设数据量不均衡 (老人2000条/店员8000条)
- 方案: 联合优化所有人设, 损失中加入组正则项: $\lambda_{\text{group}} * (w_i - \text{mean}_w)$
- 效果: 让相似人设的权重互相借力, 防止小样本过拟合

五维奖励特征

特征	计算方式	物理学上的含义
safety	$(1 - \text{smoke_pos}) \times \min(\text{fire_dist}/50, 1)$	当前位置远离烟雾和火源
efficiency	$1/(1 + \text{nearest_exit_dist}/50) \times \text{speed}$	能快速到达最近出口
social	1.0(救人), 0.5(跟从), 0.1(无)	利他行为倾向
conformity	1.0(从众), 0.5(合作), 0.3(自主)	从众程度
comfort	$(1 - \text{fear}) \times \text{speed_comfort}$	选择舒适的移动方式

离散MDP构建流程

轨迹数据(4000条) → 提取每步的特征向量 → 5维×3分箱 → ≤243个离散状态

→ 统计状态转移 $P(s'|s,a)$ → 统计专家状态-动作访问频率 $\mu_E(s,a)$

→ Soft Value Iteration → 当前w下的策略 → 策略特征期望 $\mu_{\pi}(w)$

→ 梯度更新 $w = w + lr * (\mu_E - \mu_{\pi} - l2 * w - group * (w - \text{mean}_w))$

学到的权重 (4000条Oracle轨迹, 2026-07-03)

	safety	efficiency	social	conformity	comfort
untrained_elderly	0.860	0.087	0.026	0.026	0.001
untrained_young	0.805	0.001	0.025	0.025	0.144
trained_staff	0.750	0.001	0.024	0.024	0.200

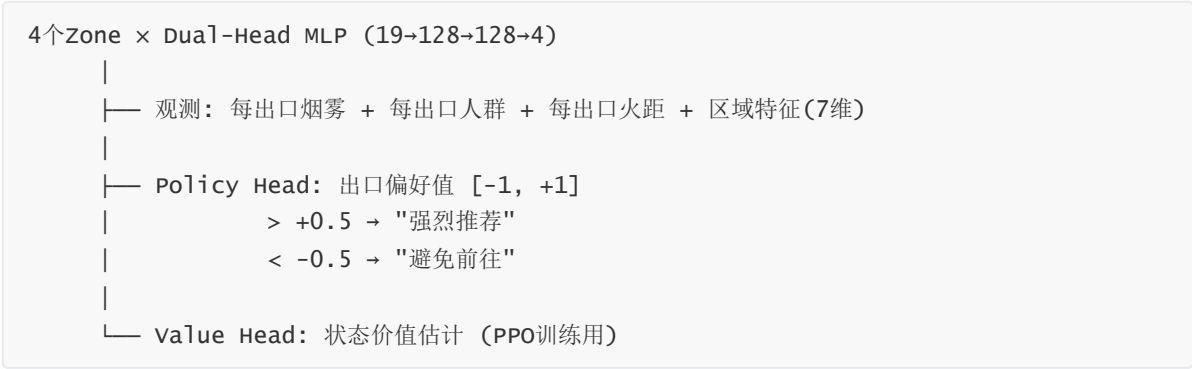
分析:

- safety权重的绝对主导 (0.75-0.86) 反映了Oracle"安全第一"的决策策略
- 老人的safety权重最高(0.86)、comfort最低(0.001), 反映"命比舒服重要"
- 年轻人的comfort权重显著高于老人(0.144 vs 0.001), 可能是走得动所以不太在意

4.3 RL区域调度器 — execution/rl_scheduler.py

在三级架构中的位置: IRL权重 → RL奖励 → 区域调度建议 → LLM Prompt

架构设计



为什么是区域级调度而非个体级调度？

个体级调度	区域级调度 (选用)
600个策略网络，每个控制一个Agent	4个策略网络，每个控制一个Zone
观测：单个Agent周围状态	观测：整个Zone的宏观统计
动作：Agent下一步去哪	动作：向Zone内所有人广播建议
训练：600个策略同时学习，不稳定	4个策略联合训练，收敛快
保留了LLM的最终决策权	✓ — 建议≠命令

启发式回退 vs 训练后MLP

当前状态： `_pretrained_loaded = False`，执行启发式逻辑：

```
# 使用IRL学到的权重做启发式出口评分
w_safety, w_efficiency = 0.805, 0.001 # 从IRL学来

for each_exit:
    safety_score = (1-smoke)*0.5 + fire_dist*0.5
    crowd_penalty = crowd_count / avg_crowd
    score = w_safety * safety_score + w_efficiency * (1-crowd_norm)
```

训练后（需要GPU服务器跑PPO）：

```
# MLP前向：19维观测 → 128(ReLU) → 128(ReLU) → 4维出口偏好
action_mean = tanh(h2 @ w3_a + b3_a) # [-1, +1] × 4出口
```

4.4 NL转换器 — `perception/nl_converter.py`

在三级架构中的角色：数值传感器 → LLM可读中文 → 决策输出

这是"跨模态对齐"的关键模块。把128×64×5的数值网格转为一个600字的中文段落。

生成的Prompt结构（一条完整LLM输入）

[环境状态]
时间：45秒 | 位置：(78.5, 32.1)
烟雾：烟雾较浓（浓度52%） | 温度：灼热（142℃）
建筑结构：完好

[火势态势]
火源位置：(100, 60) – 在你的东南方向
距你：35m | 蔓延速度：0.25m/s
需关注，预计140秒后火势到达

[出口对比分析]
出口1：距你42m | 步行约50s | 通畅 | 距火源85m | 安全 | 较少人选择
出口3：距你28m | 步行约33s | 有烟雾(45%) | 距火源18m | ⚠️高危 | 约18人选择
出口5：距你55m | 步行约65s | 通畅 | 距火源70m | 安全 | 较少人选择

【西南区调度中心建议】 ← RL调度模块注入
[推荐] 建议考虑 出口1、出口5
[警告] 避免前往 出口3（严重拥堵或危险）

[个人状态]
Agent_023：35岁店员 | 环境熟悉度：熟悉 | 体力：充沛（78/100）
心理状态：紧张 | 当前行动：walk

[最近记忆]
- [10s] 决定前往出口1：距离较远但安全
- [0s] ⚠️ 安全修正：目标出口烟雾超过安全阈值

关键设计决策

- 离散化而非原始数值：** smoke=0.52 → "烟雾较浓"，而不是直接喂 0.52。LLM对自然语言的推理比浮点数好得多。
- 出口对比而非列表：** 给6个出口的完整对比，显式标注风险标签（⚠️ 高危、🚫 已封锁），让LLM做选择题而非问答题。
- 预测信息：** 不仅告诉当前烟雾，还告诉"预计140秒后火势到达"——这部分依赖于对火蔓延速度的数值计算，LLM自己算不出来。

4.5 Tactical Layer (Brain-Torso双进程) — execution/tactical_layer.py

核心理念： 把"思考"和"反应"分开。

Slow System (Brain / LLM):

- 每10-30秒触发一次
- 只服务critical Agent (~2-5% per tick)
- 做战略决策: 选哪个出口、要不要等人、听不听引导员

Fast System (Torso / TacticalLayer):

- 每0.1秒 (每tick) 执行一次
- 服务所有Agent
- 做战术调整: 加速/减速、切换被封锁的出口

为什么需要这个: LLM推理需要50-200ms, 如果每个Agent每tick都等LLM回应, 600个Agent全部冻在原地。Tactical Layer保证Agent在"等LLM思考"时也不会站着等死。

Criticality筛选逻辑

```
def is_critical(agent):
    if has_new_info:                return True # 首次决策
    if smoke_at_pos > 0.5:          return True # 身处浓烟
    if fire_distance < 30m:         return True # 火势逼近
    if target_exit_smoke > 0.7:     return True # 出口被封
    if time_since_last_llm > 2*interval: return True # 定期刷新
    return False # 安全稳定的Agent不需要LLM
```

通常每tick只有~10-30个Agent触发LLM (600个中的2-5%), 大幅降低了GPU负载。

4.6 Cognitive Engine (LLM推理引擎) — decision/cognitive_engine.py

异步批量推理管线的四个阶段

```
[Producer] Orchestrator主线程
|
| submit_batch(agents)
▼
[Buffer] _pending_agents dict (去重)
|
| _process_next_batch()
├─ 应用chat template (Prefix Caching自动生效)
├─ 训练vLLM generate()
|
▼
[GPU Worker] daemon thread
| _run_inference()
| outputs = llm.generate(prompts, sampling_params)
| 存入 _results dict
|
▼
[Consumer] 下一tick collect_results()
|
├─ 自动chain到 _process_next_batch()
   如果GPU忙时又有新Agent加入, 不会丢失
```

失效安全机制

```
# 解析LLM输出JSON，失败则走回退
_parse_decision(raw_text) → 成功 → DecisionResult
                                → 失败 → _fallback_decision()
                                # score = dist + smoke*500
                                # 保证Agent永远有决策可执行
```

五、实验设计

5.1 三组对照

条件	LLM	IRL	RL调度	含义
SFM	✗	✗	✗	纯社会力模型基线
Pure LLM	✓	✗	✗	LLM认知能力但没有全局调度
Ours	✓	✓	✓	完整LLM→IRL→RL三级联级

5.2 期望结果

```
Ours > Pure LLM > SFM

其中：
Pure LLM > SFM:    LLM带来了角色差异化和预测推理能力
Ours > Pure LLM:   IRL→RL的全局调度避免了所有人涌向同一出口
```

5.3 统计检验

- **Cohen's d**: 效应量, >0.8为大效应
- **Mann-Whitney U**: 非参数检验, 不假设正态分布
- 每条件30 runs, 总计90次仿真

六、诚实讨论：已知局限

1. Brain-Torso架构的矛盾

所有Agent在等LLM结果前已经拿到启发式决策并执行了。LLM的"战略选择"实际上只是在追赶启发式，而TacticalLayer每tick又可以覆盖两者。

2. 三个地方用同一个公式

SFM物理、TacticalLayer启发式、LLM回退三层全部使用 `score = dist + smoke × penalty` 的变体。LLM即使想"聪明地决策"，最后也可能被覆盖。

3. 对称场景不利于区分

中心火源 + 对称出口意味着"最近出口"本身就是最优解，LLM的额外推理没有发挥空间。考虑增加**非对称火源**（角落起火）来制造真正的"绕路 vs 抄近路"困境。

4. Oracle数据的定位

Oracle不是"真实人类数据"，是对IRL算法的sanity check。论文中应诚实地定位为"算法验证实验"，而非声称学到了真实人类偏好。

5. RL策略未完成训练

`_pretrained_loaded` 仍为False，当前RL走的是启发式回退。需要在GPU服务器上完成PPO离线训练后，Ours和Pure LLM才能有实质区别。

七、当前状态 & 下一步

✔ Oracle轨迹生成	: 20 runs × 200 trajectories = 4000条, 41240+ 决策
✔ IRL权重学习	: 3 persona类别, safety主导 (0.75-0.86)
✔ 管线代码接通	: Oracle → IRL → RL → NL → LLM 全链路
⌚ RL离线训练	: 需GPU服务器, 500 episodes, ~20分钟
⌚ 完整实验	: 3条件 × 30 runs, ~12小时 (GPU服务器)

GPU服务器执行步骤

```
# Step 1: 训练RL策略
python -m execution.rl_scheduler --mode train \
    --irl_weights data/irl_weights.json \
    --episodes 500 \
    --output data/rl_policy.json

# Step 2: 跑完整实验
python -m experiments.paper_experiment \
    --config config/mall_floorplan_hard.yaml \
    --n_runs 30 \
    --output data/experiments/
```

附录：文件清单

文件	行数	角色
execution/orchestrator.py	1263	主循环，集成所有模块
execution/irl_recovery.py	782	IRL权重学习 (GC-MaxEnt)
execution/rl_scheduler.py	1247	RL区域调度 (P-MAPPO)
execution/tactical_layer.py	266	Brain-Torso快系统
perception/nl_converter.py	299	数值→自然语言
decision/cognitive_engine.py	445	vLLM异步批量推理
experiments/generate_oracle_trajectories.py	568	Oracle轨迹生成
experiments/paper_experiment.py	287	论文对照实验
execution/reward_analysis.py	533	IRL权重分析与可视化